

CSE 509 — Spring 2003

Introduction to Design Patterns

featuring the State Pattern

Lecture 7

Lois Delcambre and Andrew Black

original,
well-known
book introducing
design patterns

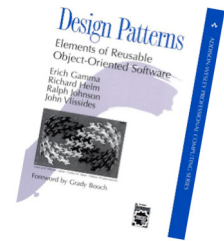
Design Patterns

Elements of Reusable
Object-Oriented Software

Examples
presented in
C++ (and
Smalltalk)

by
Erich Gamma, Richard Helm,
Ralph Johnson, John Vlissides

Addison-Wesley, 1995.



often called
the “Gang of Four”
or GoF book



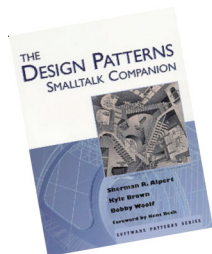
2

another resource...
follows GoF
book format

The Design Patterns Smalltalk Companion

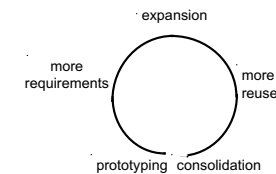
by
Sherman R. Alpert, Kyle Brown, Bobby Woolf
Foreword by Kent Beck

Addison-Wesley, 1998.



3

Reusable Software goes through an Evolutionary Cycle



What happens during the cycle?

4

In the beginning ...

- Class hierarchy reflects the problem domain
- Evolution:
 - introduces new classes
 - causes software to reflect multiple problem domains
 - causes classes to accrue unrelated responsibilities

5

Eventually...

- we must refactor the code into:
 - general purpose components
 - special purpose components

Evolution and refactoring recur cyclically

6

Why do Patterns help?

- good designers are aware of the changes that can prompt refactoring
- good designers know the object structures that help avoid refactoring
- design patterns book captures many patterns that result from software generalization
- using these patterns early helps to delay/avoid refactoring

[Contrast this with the Extreme Programming view of Refactoring]

7

Hot Spots and Cold Spots

- Rebecca Wirfs-Brock and others recommend that you identify which of your CRC cards are hot spot cards and which are cold spot cards
 - hot = very likely to change
 - cold = not very likely to change
- hot spots need patterns!

8

Common Causes of Redesign

- Creating an object by specifying a class explicitly
`new CourseOffering`
- Violating the Law of Demeter
`self.referenceSection books`
`includesKey: aBook title`
- Dependence on object representations
using `instanceOf` or `isKindof`:
- In general: depending on the detail of other objects

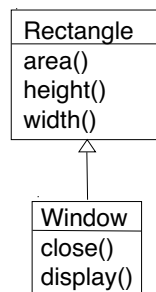
9

Advice from the Gang of Four

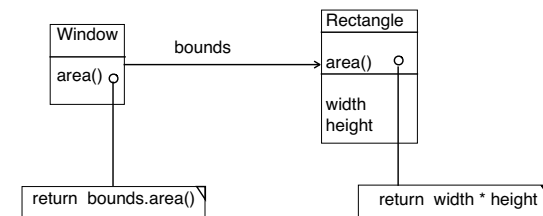
- Program to an interface, not an implementation
- Then you can't *possibly* depend on the implementation details or specific class of the object
- Favor object composition (delegation) over class inheritance
- Encapsulate the concept that varies

10

Misuse of Inheritance



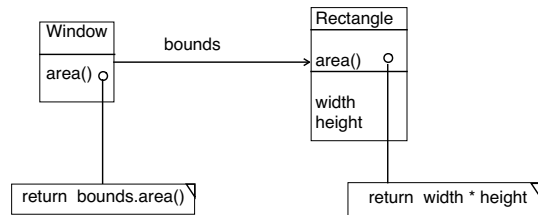
Example of delegation



Now we have two objects:
a Window object and a Rectangle object

12

Example of delegation



Let a window have a rectangle (as a bounding box) rather than be a rectangle (through inheritance)

If bounding "box" becomes a closed spline...then Window would just have a circle

13

Design Patterns provide:

- abstractions for reasoning about designs
- a common design vocabulary
- a documentation and learning aid
- leverage, based on the experience of experts
- clues to identify helper objects

They simplify the transition from analysis to implementation

14

A pattern has four essential elements:

- **pattern name** — to describe a design problem, its solution, and consequences.
- **problem** — to describe when to apply the pattern. It may include a list of conditions that must be true to apply the pattern.
- **solution** — to describe the elements that make up the design, their relationships, responsibilities, and collaborations
- **consequences** — the results and trade-offs of applying the pattern

15

Design Patterns

(highlighted patterns are covered in the lecture)

scope		Purpose		
		Creational	Structural	Behavioral
class	factory method		adapter	interpreter template method
	object	abstract factory builder prototype singleton	adapter bridge composite decorator facade flyweight proxy	chain of resp. command iterator mediator memento observer state strategy visitor

16

The State Pattern

- Intent:** Allow an object to alter its behavior when its internal state changes.
Object appears to change class.
- Applicability:** When an object's behavior depends on its state and it must change its state-dependent behavior at run-time.
When operations have large, multi-part conditions that depend on the object's state
- Technique:** Introduce an abstract class called State
Introduce one concrete subclass of State for each state
Create one object of each subclass.
Context has a state — delegates behavior to state
The state of the context is as an object in its own right

17

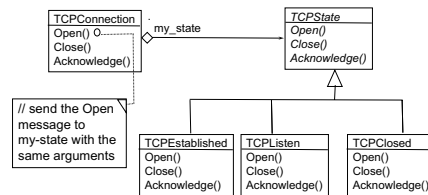
State Pattern

Allows object to change behavior when its internal state changes (object will appear to change its class)

1. Introduce an abstract class called "State"
2. Use concrete subclasses to represent the possible states
3. Define all operations in the abstract class State
4. Then override them in the states, as appropriate (certain operations can be ignored... or produce errors)

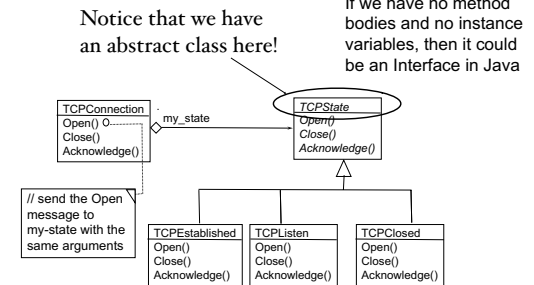
Example: managing the state of a TCP/IP connection

18



If the TCPConnection changes state, then it simply replaces the object of one state with an object of another state

19

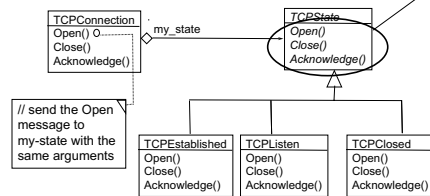


Notice that we have an abstract class here!

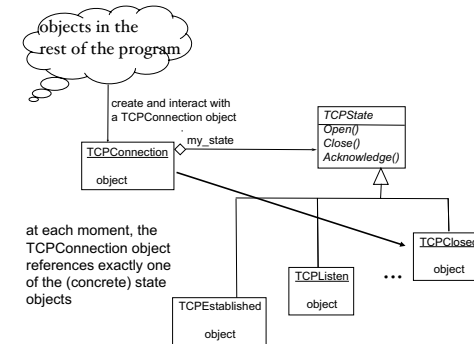
If we have no method bodies and no instance variables, then it could be an Interface in Java

20

Notice that we have abstract methods here! Alternatively, they could provide default behavior (usually “do nothing”)

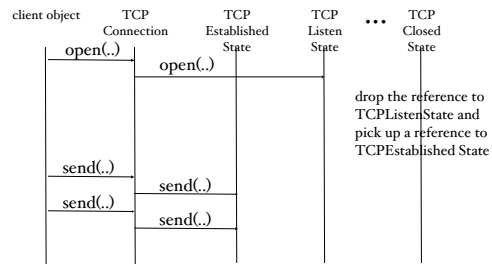


21



22

Sequence Diagram



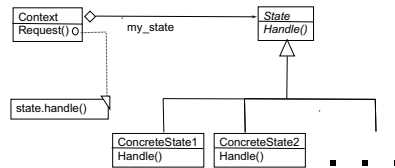
23

Design Decisions for the State Pattern

- How/when are the state objects created? How are they addressed?
- Are the state objects shared?
- Who is responsible for making the state transitions? Methods in the concrete states? Or methods in the **TCPConnection** objects?
- Is “**TCPState**” an interface or an abstract class?
- where will the methods where the work is actually accomplished be? In the concrete states? in the **TCPConnection**?

24

Generic Class Diagram for the State Pattern



25

The State Pattern

Participants:

- Context (TCPConnection)** — defines the interface used by clients
- State (TCPState)** — defines an interface for encapsulating the behavior for a state
- ConcreteState (TCPEstablished, TCPListen, TCPClosed)** — each subclass implements a behavior associated with a state

Collaborations:

- Context delegates state-specific behavior to ConcreteState object
- Context may pass itself as an argument to ConcreteState object
- Context is the primary interface for clients once states are configured. Clients unaware of ConcreteState objects
- Either Context or concrete state subclasses can decide which state succeeds another, and when

26

The State Pattern

Consequences: Localizes state-specific behavior. Partitions behavior for different states. New states & transitions can be added easily.

Reifies state transitions: the context actually “has” a different state.

State objects can be shared (if they provide only state-specific behavior). All objects in the same state can “have” the same (singleton) state object.

27

Smalltalk Example of TCP Connection

The Design Patterns Smalltalk Companion by Alpert et al.

```

Object subclass: #TCPConnection
  instanceVariableNames: 'state'
  classVariableNames: ''
  poolDictionaries: ''

```

```

Object subclass: #TCPState
  instanceVariableNames: ''
  classVariableNames: ''
  poolDictionaries: ''

```

```

TCPConnection>>activeOpen
  "delegate the open message to the current state."
  self state activeOpen: self

```

28

Smalltalk Example of TCP Connection (cont.)

```
Object subclass: #TCPConnection
  instanceVariableNames: 'state'
  classVariableNames: ''
  poolDictionaries: ''
```

```
Object subclass: #TCPState
  instanceVariableNames: ''
  classVariableNames: ''
  poolDictionaries: ''
```

```
TCPConnection>>activeOpen
  "delegate the open message to the current state."
  self state activeOpen: self
```

send it the activeOpen message (with self as an argument)

get current state object

29

Smalltalk Example of TCP Connection (cont.)

```
TCPState>>activeOpen: aTCPConnection
  "Don't implement an open method: expect the concrete subclasses to"
  self subclassResponsibility
```

and do the same thing for all the messages for TCPState (that is, TCPState is a completely abstract class)

```
TCPState subclass: #TCPEstablished
  instanceVariableNames: ''
  classVariableNames: ''
  poolDictionaries: ''
```

and do the same thing for all other concrete states that you need (TCPListen state, TCPClosed state, etc.)

30

Smalltalk Example of TCP Connection (cont.)

```
TCPEstablishedState>>activeOpen: aTCPConnection
  "Do nothing....the connection is already open"
  ^self
```

```
TCPClosedState >>activeOpen: aTCPConnection
  "do the open—invoke the establishConnection method of TCPConnection"
  ^aTCPConnection establishConnection
```

```
TCPConnection>>establishConnection
  "Do the work to establish a connection. Then change state."
  self state: TCPEstablishedState new
```

31

Smalltalk Example of TCP Connection (cont.)

```
TCPEstablishedState>>activeOpen: aTCPConnection
  "Do nothing....the connection is already open"
  ^self
```

```
TCPClosedState >>activeOpen: aTCPConnection
  "do the open—invoke the establishConnection method of TCPConnection"
  ^aTCPConnection establishConnection
```

```
TCPConnection>>establishConnection
  "Do the work to establish a connection. Then change state."
  self state: TCPEstablishedState new
```

send the state: message to self to assign a new object

create a new TCPEstablishedState object

32

Design Decisions for the Smalltalk example

- How/when are the state objects created? how are they addressed?
- Are the state objects shared?
- who is responsible for making the state transitions? methods in the concrete states? or methods in the TCPConnection objects?
- is "TCPState" an interface or an abstract class?
- where will the the work actually be performed? in the concrete states? in TCPConnection?

33

Design decisions for the Smalltalk example

- How/when are the state objects created? Every time we make a state transition! How are they addressed? returned by new operator
- Are the state objects shared? No
- Who is responsible for making the state transitions? methods in the concrete states? or methods in TCPConnection? State transitions are made in TCPConnection (within the methods that actually perform the valid operations)
- Is "TCPState" an interface or an abstract class? TCPState is an abstract class (Smalltalk doesn't have a syntax for interfaces)
- Where will the the work actually be performed? in the concrete states? in TCPConnection? In methods of TCPConnection

34

Java Example of the State Pattern

loosely based on ~~from~~ *Patterns in Java, Volume 1* by Mark Grand

```
public class TCPConnection {  
  
    private TCPState myState;  
  
    // objects for states  
    private static final TCPClosed    tcpclosedstate    = new TCPClosed();  
    private static final TCPOpen      tcpopenstate      = new TCPOpen();  
    private static final TCPEstablished tcpeestablishedstate = new TCPEstablished();  
  
    public static void main ( String [ ] args )  
    {  
        TCPConnection tc = new TCPConnection();  
  
        System.out.println( "Starting TCP example" );  
        tc.myState.processEvent(tc, ConnectionEvent.Open);  
        tc.myState.processEvent(tc, ConnectionEvent.Close);  
        tc.myState.processEvent(tc, ConnectionEvent.Open);  
        tc.myState.processEvent(tc, ConnectionEvent.Connect);  
        tc.myState.processEvent(tc, ConnectionEvent.Connect);//should throw exception  
    }  
}
```

35

Java Example of the State Pattern

(cont.)

```
interface TCPState {  
    void processEvent(TCPConnection tc, ConnectionEvent event);  
    void enter(TCPConnection tc);  
}  
  
private static class TCPClosed implements TCPState {  
    public void processEvent(TCPConnection tc, ConnectionEvent event) {  
        String msg;  
  
        if (event == ConnectionEvent.Open) {  
            tcpopenstate.enter(tc);  
            return; }  
        else if (event == ConnectionEvent.Send) {  
            msg = "unexpected event " + event;  
            throw new IllegalArgumentException(msg); }  
        else if (event == ConnectionEvent.Close) {  
            return; }  
        else /* event == Connect */ {  
            msg = "unexpected event " + event;  
            throw new IllegalArgumentException(msg); }  
    }  
}
```

36

Java Example of the State Pattern

(cont.)

```
public void enter(TCPConnection tc) {  
    // Do whatever is needed to close tc.  
    System.out.println( "Closing..." );  
    tc.myState = tcpclosedstate;  
}  
}
```

37

Design Decisions for the Java example

- How/when are the state objects created? When the class TCPState is initialized (the static variables are initialized to new objects) How are they addressed?
- Are the state objects shared?
- Who is responsible for making the state transitions? Methods in the concrete states? Or methods in the TCPConnection objects? Methods in the concrete states
- Is "TCPState" an interface? an abstract class? TCPState is an interface
- Where are the methods where the work is actually accomplished) be performed? In the concrete states? In TCPConnection? In the "enter" method for each concrete state

38

Design Decisions for the Java example

- How/when are the state objects created? When the class TCPState is initialized (the static variables are initialized to new objects) How are they addressed? In static variables of TCPState
- Are the state objects shared? Yes
- Who is responsible for making the state transitions? Methods in the concrete states? Or methods in the TCPConnection objects? Methods in the concrete states, which can access the field of the parent
- Is "TCPState" an interface? an abstract class? TCPState is an interface
- Where are the methods where the work is actually accomplished) be performed? In the concrete states? In TCPConnection? In the "enter" method for each concrete state

39

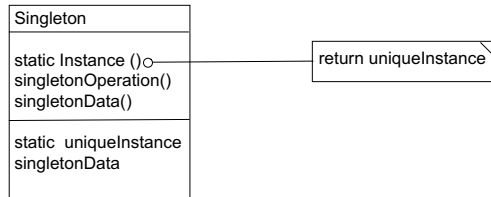
The Singleton Pattern

- **Intent:** Ensure a class has only one instance. Provide a global point of access to the instance
- **Motivation:**
 1. Make sure a class has only one instance despite anything that a client might do
 2. Make the class responsible for keeping track of its sole instance
- **Applicability:** when there must be exactly one instance, and it must be globally accessible.

40

The Singleton Pattern

Structure:



41

The Singleton Pattern

- Participants:
Singleton defines an instance() method. It is responsible for creating its own unique instance
- Collaborations:
Clients obtain access to singleton solely through its instance() method

42

The Singleton Pattern

- Consequences:
 1. Controlled access to sole instance
 2. Reduced name space (no need for global variable)
 3. Singleton class could have subclasses and, through delegation, you can easily configure with the appropriate subclass (like in the State pattern)
 4. Could be changed to limit to a specific number of instances (e.g., doubleton)

43

The Singleton Pattern: Smalltalk Implementation

- In Smalltalk, the method that returns the unique instance is a class method on the Singleton class. The new method is cancelled.
- SoleInstance is a class variable.
- There are two class methods:

```

new
  self error: 'cannot create new instances of ', self name

uniqueInstance
  SoleInstance ifNil: [SoleInstance := super new initialize].
  ^SoleInstance
  
```

44

The Singleton Pattern: Java Implementation

Implementation:

```
public class Singleton
{
    private static Singleton instance = new Singleton();

    private /* constructor */ Singleton() { }

    public static Singleton uniqueInstance() {
        return instance;
    }
}
```

45

Design Decisions for the Java example

- What stops a client from creating more than one “Singleton” ?
- How does a client obtain access to the singleton object?

46